

# teamscale-mcp & teamscale-docs-mcp

[github.com/MarcelBruckner/teamscale-mcp](https://github.com/MarcelBruckner/teamscale-mcp) · **teamscale-docs-mcp**

*Two sidecars: the full REST API plus the Teamscale docs, for any client and any identity*

## Three pieces, three jobs

-  **Teamscale Claude Code plugin**: curated skills for developers editing code
-  **teamscale-mcp**: the full REST API as tools (the data)
-  **teamscale-docs-mcp**: the Teamscale docs as tools (the understanding)

The plugin helps a developer change the code Teamscale analyses. These two help us, and the customers we work with, read, explain, and configure Teamscale itself.

**This talk is about that second pair.**

## Quick recap: published plugin

The official `teamscale/claude-code` plugin:

- Per-user and Claude-Code-only, installed on each workstation
- Bundles curated skills ( `fix-findings` , `pr-*` , `local-*` ) ...
- ... plus a local backend ( `ts_agent_helper` ) that those skills call
- Needs Python, the `teamscale-dev` CLI, a `.teamscale.toml` , and a local checkout

Its value is **opinionated workflows for a developer fixing findings** in the analysed code, with the repo checked out on their own machine.

## What **teamscale-mcp** is

One central HTTP sidecar, deployed once next to Teamscale:

- *One tool per REST endpoint*, generated at startup from the instance's live OpenAPI spec
- *Per-request identity* via `X-Teamscale-User` / `-Token` headers
- Reachable by *any MCP client* over a URL

If the API can do it, there's a tool for it, and it always matches the instance's own version.

## Configured entirely by **env vars**

teamscale-mcp:

```
image: ghcr.io/marcelbruckner/teamscale-mcp:latest
ports: ["8081:8081"] # serving: MCP_HOST / _PORT / _PATH, MCP_ALLOWED_HOSTS
environment:

# Connection + bootstrap creds (used once, at startup, to fetch the spec)
TEAMSACLE_SERVER_URL: http://teamscale:8080
TEAMSACLE_SPEC_USER: some-technical-user
TEAMSACLE_SPEC_TOKEN: some-technical-user-api-token
TEAMSACLE_INCLUDE_INTERNAL: "false"

# Tool surface: trim to a sharp, read-oriented set (blocklist recommended)
TEAMSACLE_EXCLUDE_TAGS: "Backup,System,Users,Profilers,SAP,..."
TEAMSACLE_EXCLUDE_METHODS: "DELETE,PUT,PATCH"
# TEAMSACLE_INCLUDE_TAGS: "Findings,Metrics,Test Gap Analysis"
# TEAMSACLE_INCLUDE_NAMES: "createBaseline"
```

Per-request identity arrives in the `X-Teamscale-User` / `-Token` headers, never from the environment.

## Connect a client with **one command**

Point any MCP client at the URL, presenting your own identity:

```
claude mcp add teamscale-cqse --scope user --transport http \
  https://cqse.teamscale.io/mcp \
  --header "X-Teamscale-User: bruckner@cqse.eu" \
  --header "X-Teamscale-Token: YOUR_API_TOKEN"
```

The two headers *are* your Teamscale identity, forwarded per request.  
Any MCP-capable client works the same way.

## Connect a client with **one command**

Point any MCP client at the URL, presenting your own identity:

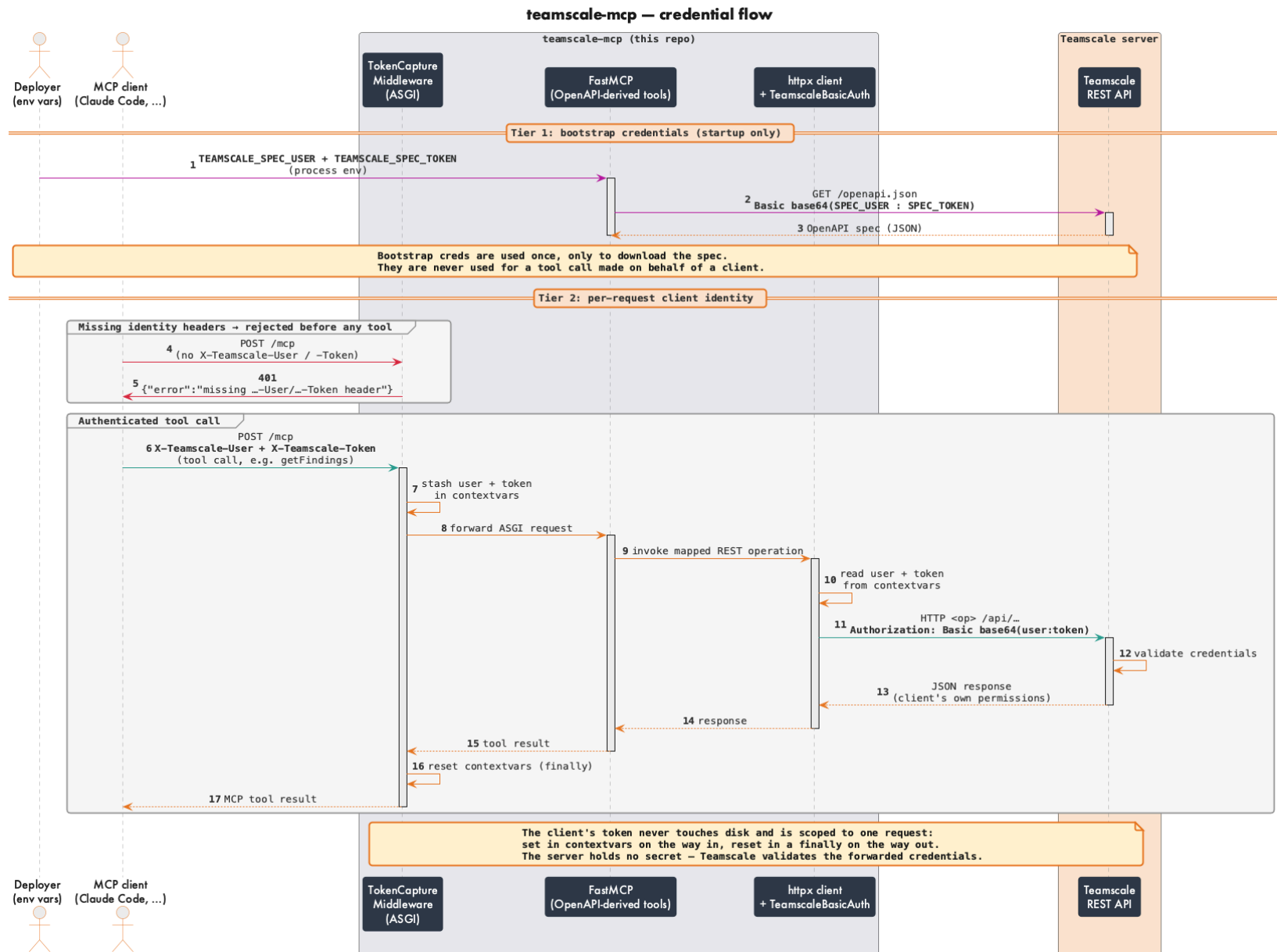
```
claude mcp add teamscale-cqse --scope user --transport http \  
  https://cqse.teamscale.io/mcp \  
  --header "X-Teamscale-User: bruckner@cqse.eu" \  
  --header "X-Teamscale-Token: YOUR_API_TOKEN"
```

And add as many servers as you need:

```
claude mcp add teamscale-personal --scope user --transport http \  
  https://teamscale.marcelbruckner.com/mcp \  
  --header "X-Teamscale-User: marcel" \  
  --header "X-Teamscale-Token: YOUR_API_TOKEN"
```

The two headers *are* your Teamscale identity, forwarded per request.  
Any MCP-capable client works the same way.

## The credential flow: two tiers, no stored secret



## The honest reframe: what it is *not*

It is **not** a drop-in replacement for `ts_agent_helper`:

- Only 3 of 6 skills route through it
- `fix-findings` is a clean win (pure REST)
- `pr-*` is a wash, because the agent has to redo the git/PR resolution the CLI bundled
- `local-*` can't move at all, since they need the local working tree

"Replaces `ts_agent_helper`" is the weakest way to sell it. The next slides are the strong way.

## Where **teamscale-mcp** really fits

The real value is reach the per-user plugin can't offer:

- 🔍 **Data access** across the entire API, for questions no skill encodes
- 🏢 **One central deployment** per instance with no per-workstation Python
- 📈 **Observability**: one server, and every tool call is routed through it
- 🤖 **CI and service accounts**: an HTTP endpoint any client can call, as a technical account rather than a person's login
- 🧠 **Any LLM integration**: plain MCP over HTTP, so it's not tied to Claude Code; any MCP-capable client or model can use it
- 🧑 **Weakest fit**: an individual dev's daily fix loop, where you'd just use both

## A different **audience**, not just a different deployment

The sharpest distinction isn't *how* it's run, it's *who* it's for.

|           | Plugin                     | teamscale-mcp (+ docs)         |
|-----------|----------------------------|--------------------------------|
| Works on  | the analysed code          | the Teamscale data & config    |
| Assumes   | a checkout in front of you | just a URL and an identity     |
| Built for | developers editing code    | people understanding Teamscale |

- Us on customer instances
  - The customer managers we report to
  - and newcomers setting an instance up
- People who read Teamscale's results and act on them, without shipping the analysed code.

## **teamscale-docs-mcp**, the understanding half

Teamscale's documentation, served as MCP tools:

- A *sitemap catalog*, plus individual pages converted to Markdown on demand
- *No auth*, because the docs are public content
- Points at an instance's *version-matched bundled docs*, or at the public site
- Self-hosted alongside Teamscale, or run as one central endpoint

It hands the agent Teamscale's manual, not just its data.

## Configured by a single **env var**

teamscale-docs-mcp:

```
image: ghcr.io/marcelbruckner/teamscale-docs-mcp:latest
ports: ["8082:8082"]           # serving: MCP_HOST / _PORT / _PATH, MCP_ALLOWED_HOSTS
environment:

# The only real setting: where the docs live. Point at the instance's
# version-matched bundled docs, so no request leaves the network ...
DOCS_BASE_URL: http://teamscale:8080/documentation

# ... or serve the public site instead:
# DOCS_BASE_URL: https://docs.teamscale.com
```

No credentials, no headers: the docs are public content, so there is nothing to authenticate.

## Explain and act: the pairing that matters

Neither half is the point on its own. Together they are.

-  **teamscale-docs-mcp** explains: concepts, how-tos, reference
-  **teamscale-mcp** acts: read data, set up projects, adjust analysis profiles

In one conversation you get a data-and-understanding assistant, exactly what the code-editing plugin can't be.

## Generic docs, tailored to *your* repo

- 📖 **teamscale-docs-mcp**: the generic how-to (any language, any stack)
- 🔌 **teamscale-mcp**: *this* project's real languages, test framework, build/CI config
- ✍️ Claude merges the two into a runbook written for this repo specifically

Same how-to page, different repo → a different, correct set of steps every time, not a generic doc dump the reader has to adapt themselves.

## What "explanation" actually looks like

-  **A customer's manager:** *"This test gap is 34%. Is that bad, and where did it come from?"*  
The docs explain the concept, and teamscale-mcp pulls the number and trend.
-  **New to Teamscale:** *"How do I set up an analysis profile for Kotlin?"*  
The docs give the how-to, and teamscale-mcp creates and wires it.
-  **Us, on a customer instance:** *"Compare clone coverage across these five projects."*  
Full REST reach, with the reference at hand, across customer instances.

Understand first, act second, always grounded in the customer's real data.

## A real session: TGA + frontend profiling

Not a mockup, but an actual conversation against a live project:

- Learn what Test Gap Analysis *is* (docs)
- Pull its real number (teamscale)
- Get a setup runbook for frontend coverage (docs), tailored to this project's stack

Docs and data interleaved end to end.

I want to know about TGA in Teamscale

• I'll look this up in the Teamscale documentation rather than rely on memory. Let me load the documentation tools.

Called teamscale-marcelbruckner-com-docs 4 times

• TGA = Test Gap Analysis – one of Teamscale's core "Test Intelligence" features. Here's what it is and how it works.

The problem it solves

Most production bugs occur in **recently changed code**. But testers often don't know everything that changed – refactorings, undocumented bug fixes, or whole features get shipped without the test team being told. The result: risky changes go to production untested. (CQSE's own research found this accounts for the majority of production defects.)

What TGA does

Test Gap Analysis combines three things Teamscale already tracks – **code changes**, **test coverage**, and **issues** – to visualize *every* recently changed method and whether it has been tested yet.

The signature view is the **Test Gap treemap**, where each rectangle is a changed method:

- Gray – unchanged code
- Green – changed *and* tested
- Red / orange – changed but **never tested** → these are your *Test Gaps*

This lets testers spot untested risk and schedule additional test cases *before* release.

Key capabilities

- Works across all **test stages**, not just unit tests: integration, end-to-end, manual, and even hardware/software-in-the-loop tests. Many teams see the true coverage of these non-unit stages for the first time.
- **Issue-level TGA** – aggregates test gaps per Jira issue / Azure DevOps work item, so a tester can see exactly how much of the code changed for a given story/feature is still untested, and filter/prioritize accordingly.
- **Pull-request integration** – Teamscale can post PR comments flagging open test gaps, putting the info directly in front of developers.
- **KPIs & trends** – long-term coverage trend per stage, unaddressed gaps per PR, coverage per user story/defect/epic. Supports a "shift-left" feedback loop.

Getting started (relevant docs)

| Task                             | Page                                                                 |
|----------------------------------|----------------------------------------------------------------------|
| Record coverage via the profiler | /howto/setting-up-profiler-tga/ (Java, C#, C++, JS, Node, Python...) |
| Coverage for manual tests        | /howto/setting-up-profiler-tga/manual-tests/                         |
| Reading the treemaps             | /howto/working-with-test-gap-treemaps/                               |
| Issue-level test gaps            | /howto/analyzing-issue-test-gaps/                                    |
| Build a TGA dashboard            | /howto/creating-a-tga-dashboard/                                     |

| Note: TGA (Test Gap Analysis) is distinct from Teamscale's smart **test suggestions** (Test Impact Analysis / Pareto) – those focus on which tests to run fast, whereas TGA focuses on which changes are untested.

Want me to walk through any specific part – e.g. setting up the profiler to record coverage, reading the treemap, or issue-level test gaps?

\* Cogitated for 37s

18

## A real session: TGA + frontend profiling · 2. pull the number (first try)

I want to know about TGA in my traureise project

Called teamscale-marcelbruckner-com

- Found three matching projects: traureise, traureise-fastapi-react, and traureise-original. Let me pull the TGA data for the exact traureise project.

Called teamscale-marcelbruckner-com 2 times

- Here's what I found for your Traureise project (traureise):

TGA result

Test Gap percentage: 0%

- Project: Traureise (traureise), default branch main, configuration completed.
- Computed over the full default-branch history (default baseline, all default coverage partitions).

How to read this

A TGA percentage of 0% means zero percent of the analyzed changed code is showing up as a test gap. There are two very different reasons this can happen, and it's worth confirming which one applies to you:

1. Everything relevant is covered – all changed methods in scope have been executed by tests. Genuinely 0 gaps. 🏆
2. No test coverage has been uploaded – with no coverage data, there's nothing to compare changes against, so the metric collapses to 0. This is the more common cause for a newer project.

Given the project was created fairly recently and I can't see any coverage partitions referenced, cause #2 (no coverage uploaded yet) is the likely explanation – but I'd want to verify rather than assume.

Heads up: three matching projects

There are actually three projects with "traureise" in the name:

- traureise ← this one
- traureise-fastapi-react
- traureise-original

If you meant a different one, tell me and I'll pull its numbers.

Want me to dig deeper?

I can:

- Check whether coverage exists (query with all-partitions / execution-only to see if any methods were executed) to confirm which of the two cases above applies,
- Break down the Test Gap treemap to see changed vs. tested vs. untested methods,
- Pull issue-level test gaps for specific tickets, or
- Run the same for traureise-fastapi-react / traureise-original.

Which would you like?

No, I'm sure coverage exists

- Good – then let me figure out why the default query returned 0%. My first call used only the default coverage partitions; if your coverage lives in other partitions it would be ignored. Let me re-run with all partitions considered, and separately check whether methods are being seen as executed at all.

Called teamscale-marcelbruckner-com 3 times

- All queries – including the churn-only one, which ignores coverage entirely – return 0. That's the key clue: churn TGA of 0 means Teamscale sees no *changed* methods in the analyzed window. TGA always measures "of the code that changed since a baseline, how much is untested" – so with no baseline set, the change window is effectively empty and the result collapses to 0 regardless of coverage.

Let me check the configured baselines and re-run over an explicit time window (from before the project's first commit through HEAD).

Called teamscale-marcelbruckner-com 2 times

- That's the answer. The reason every earlier query returned 0% is now clear:

Root cause

Your project has no configured baselines (`getAllBaselines` → empty). TGA measures test gaps *relative to a baseline* (the start of the change window). With no baseline set, the default query had an empty change window → 0 changed methods → 0% by definition. It was never a coverage problem – your coverage is fine, as you said.

Actual TGA for Traureise

Measuring across the full project history (from before the first commit through `main:HEAD`, all coverage partitions):

Test Gap ratio: ~22.1%

Interpreting the treemap semantics:

- ~22% of the methods changed since the project began have never been executed by a test – these are your test gaps.
- ~78% of changed methods have been covered. 🙌

Why this matters for your setup

Because there's no baseline defined, the TGA widgets/dashboards in the UI will also show 0 by default until you give them a time window. Two fixes:

1. Configure a baseline (e.g., "last release" or a fixed date) under project settings → then TGA reflects gaps *since that point*, which is the usual way teams track "did we test what we changed this release." See </howto/working-with-test-gap-treemaps/>.
2. For ad-hoc analysis, just specify a start/end time window (as I did here) rather than relying on the default.

Want me to:

- Break down the treemap to show which specific methods/files make up that 22% gap,
- Compute TGA since a specific date (e.g., a recent release) instead of all-history, or
- Check issue-level test gaps for particular tickets?

● I've localized the 22% gap down to the file level – that's as granular as the API goes (per-method red/orange cells only exist in the UI treemap). Here's the breakdown.

Where the ~22% test gap lives

Frontend – the bulk of the gap

| Path                       | Test gap |
|----------------------------|----------|
| frontend/src (entire tree) | 100%     |

None of your frontend source (components/, routes/, hooks/, etc.) maps to any recorded coverage. Even though there are Playwright specs under frontend/tests, no coverage from them is being associated with the frontend source methods in Teamscale – so every changed frontend method counts as a gap. This is the single largest contributor.

Backend – small, concentrated gap (backend/app = 13%)

| File                                                               | Test gap | Note                                              |
|--------------------------------------------------------------------|----------|---------------------------------------------------|
| backend/app/api/routes/utls.py                                     | 100%     | fully untested                                    |
| backend/app/utls.py                                                | 14.3%    | ~1 in 7 changed methods untested                  |
| backend/app/api/routes/login.py                                    | 20%      | e.g. password-recovery / reset handlers typically |
| backend/app/crud.py                                                | 0%       | fully covered ✓                                   |
| backend/app/core/*                                                 | 0%       | ✓                                                 |
| backend/app/api/routes/items.py, users.py, private.py, websites.py | 0%       | ✓                                                 |

Reading this

- Your backend is in good shape – gaps are limited to utility/helper code (utls.py, the routes/utls.py endpoints) and part of login.py. Those are the concrete methods to add tests for.
- The headline number is dominated by the frontend, which shows as 100% gap. Before writing a pile of frontend tests, it's worth checking why – most likely the frontend coverage (Playwright/Vitest) either isn't being uploaded or isn't mapped to a coverage partition Teamscale associates with frontend/src. If you expected frontend coverage to exist, that's a coverage-upload/mapping issue rather than a genuine testing gap.

Getting exact method names

The public API exposes TGA only as a percentage per path, not the individual red/orange methods. To see the exact untested methods, open the Test Gap treemap in the Teamscale UI for traureise (set the same time range / a baseline) – each red or orange cell is a specific changed-but-untested method, and clicking it jumps to the code.

Want me to:

- Drill the frontend into subfolders (components vs routes vs hooks) to rank them, or
- Check whether any frontend coverage partition exists at all (to confirm the upload/mapping theory)?

Can you help me setting up coverage profiling for frontend code?

- This is a documented setup path, so let me pull Teamscale's own frontend-coverage guides before advising – I want to give you the exact steps for your stack (Vite + React + TypeScript with Playwright e2e tests), not generic ones.
- Wrong tool name – the docs server has its own prefix. Let me use the correct one.

Called teamscale-marcelbruckner-com-docs 2 times

- I've got Teamscale's frontend-coverage guide. Your frontend is **Vite + React + TypeScript with Playwright e2e tests** (frontend/tests/\*.spec.ts, playwright.config.ts) – that's exactly the scenario the JavaScript profiler is built for: an app served to a browser, tested by automated UI tests.

Before I write you the concrete runbook, two decisions materially change the steps – let me pin them down.

- User answered Claude's questions:

- └ · Which coverage-collection approach do you want to use for the frontend? → Teamscale JS Profiler (Recommended)
- Which test stage(s) should produce the frontend coverage that feeds TGA? → Playwright E2E, Unit tests (Vitest/Jest)
- Do you have the traipse frontend checked out locally so I can edit config files, or should I give you a written runbook? → Written runbook

• Here's your runbook. Since you want **both** stages, note one important split up front:

- Playwright E2E runs in a real browser → use the **Teamscale JavaScript Profiler** (instrumenter + collector). This is the recommended path.
- Vitest unit tests run in Node/jsdom → the profiler *can't* be used there (instrumented code fails in Node). Vitest produces its own coverage, which you upload separately.

So you'll end up with **two coverage partitions** feeding TGA, e.g. E2E Coverage and Unit Test Coverage.

---

### Part A – One-time setup in Teamscale (both stages)

1. Go to **Project Configuration** → **Coverage Profilers** in the traureise project.
2. Create a profiler config, ID `traureise-e2e`:  
teamscaleProject=traureise  
teamscalePartition=E2E Coverage  
dumpAfterMins=120
2. (The Vitest path doesn't need a profiler config – it uploads directly via teamscale-upload.)

---

### Part B – Playwright E2E coverage (JS Profiler)

1. Enable source maps (so `dist/` maps back to `frontend/src`)

```
frontend/tsconfig.json:
{ "compilerOptions": { "sourceMap": true, "inlineSources": true } }
frontend/vite.config.ts:
export default defineConfig({
  build: { sourcemap: true },
})
```

2. Build and instrument the app

The profiler instruments the **built** output – you must serve the built+instrumented app, *not* the Vite dev server.

```
cd frontend
npm run build                # produces ./dist with source maps
```

```
npx @teamscale/javascript-instrumenter \
  --input ./dist/ \
  --collector localhost:54678 \
  --config-id traureise-e2e \
  --commit "$(git rev-parse HEAD)" \
  --in-place \
  --include-origin 'src/**/*'
- --commit ties the coverage to the exact revision being tested (essential for TGA).
- --include-origin 'src/**/*' matches your frontend/src sources.
```

3. Start the collector (auto-upload to Teamscale)

Add to `frontend/package.json`:

```
"scripts": {
  "collector": "coverage-collector --teamscale-server-url https://teamscale.marcelbruckner.com --teamscale-user <user> --teamscale-access-key <key> --teamscale-project traureise",
  "pree2e:cov": "npx pm2 delete CC; npx pm2 start npm --name CC -- run collector",
  "e2e:cov": "playwright test",
  "poste2e:cov": "npx pm2 delete CC"
}
```

Install: `npm install -D @teamscale/coverage-collector pm2`

### 4. Point Playwright at the instrumented build

In playwright.config.ts, make webServer serve the built dist/ (e.g. vite preview or npm run preview), not vite dev:  
webServer: { command: 'npm run preview', url: 'http://localhost:4173', reuseExistingServer: false }

### 5. Content Security Policy (only if you set one)

The FastAPI full-stack template doesn't ship a strict CSP by default, so the WebSocket to the collector should just work. If you later add a CSP, it must allow the collector:  
connect-src 'self' ws://localhost:54678;  
script-src 'self' blob: ws://localhost:54678;  
worker-src 'self' blob: ws://localhost:54678;

### 6. Run it

npm run e2e:cov  
The collector receives coverage over WebSockets as Playwright drives the browser, then uploads to the E2E Coverage partition.

---

### Part C – Vitest unit-test coverage

| If you don't have a Vitest suite yet, this stage produces nothing until you add one. Say the word and I'll help scaffold it.

#### 1. Enable coverage in Vitest

```
cd frontend
npm install -D @vitest/coverage-istanbul
vite.config.ts (or vitest.config.ts):
test: {
  coverage: {
    provider: 'istanbul',
    reporter: ['lcov'],           // lcov → Teamscale reads it directly
    reportsDirectory: './coverage',
    include: ['src/**/*.ts'],
  },
}
```

#### 2. Run and upload

```
npx vitest run --coverage
npx @cqse/teamscale-upload \
  --server https://teamscale.marcelbruckner.com \
  --project traureise \
  --user <user> --accesskey <key> \
  --partition "Unit Test Coverage" \
  --format LCOV \
  --commit "$(git rev-parse HEAD)" \
  ./coverage/lcov.info
```

---

### Part D – Verify

After each upload, TGA for the frontend should stop reading 100%. You can confirm by asking me to re-run getTgaPercentage for frontend/src (all partitions), or check the Test Gap treemap in the UI.

---

A couple of things worth flagging:

- The instrumenter must run against the built bundle each time you test a new commit – wire steps B2-B6 into your CI so coverage stays current, otherwise TGA drifts stale.
- Use git rev-parse HEAD consistently for --commit in both stages so E2E and unit coverage land on the same revision.

Want me to (a) generate a ready-to-paste CI job (GitHub Actions, matching your repo layout), or (b) help scaffold the Vitest suite if you don't have one yet?

The same pairing, on a tool outside either server

`teamscale-docs-mcp` also documents `teamscale-dev` .

`teamscale-dev` itself stays out of scope for both servers, but *explaining* it doesn't.

## The same pairing, on a tool outside either server

Check the teamscale docs. How do i configure teamscale-dev

- I'll search the Teamscale docs for how to configure `teamscale-dev`.

Called teamscale-marcelbruckner-com-docs 2 times

- Found it: `/reference/cli/teamscale-dev/`. Let me read that page.

Called teamscale-marcelbruckner-com-docs

- Let me also pull the `.teamscale.toml` reference, since that's the config file `teamscale-dev` reads.

Called teamscale-marcelbruckner-com-docs

- Here's how to configure `teamscale-dev` (the Teamscale developer CLI). There are two distinct pieces: **what to analyze** (a `.teamscale.toml` file) and **how to authenticate** (server credentials).

### 1. Project config: `.teamscale.toml`

`teamscale-dev` reads `.teamscale.toml` files (TOML format) to know which server/project/branch a directory maps to. A minimal file:

```
server.url = "https://teamscale.io/"
project.id = "example"
```

Full set of properties:

```
version = "1.0" # optional, default 1.0
root = false   # if true, config discovery stops here and doesn't look further up
```

```
[server]
url = "https://teamscale.io/" # must resolve in the merged config; must match a server you have credentials for
```

```
[project]
id = "example" # the Project ID (may differ from the display name)
branch = "main" # optional; if unset, discovered from VCS, falling back to server default
path = "remote/path" # optional; path within the Teamscale project (" " = project root)
```

Notes:

- **Merging:** files inherit from parent directories (like `.editorconfig`). Put `server.url` + `project.id` in your repo root, and override just `project.path` in subdirectories. A `root = true` file stops upward discovery.
- **Auto-generate it:** in Teamscale, go to *Project Configuration* → *Projects* and click *Configure IDE plug-ins / CLI* to download a ready-made file.

The same pairing, on a tool outside either server

2. Credentials (username + access key)

Config files hold no secrets – you supply your username and access key separately. Five ways, in precedence order:

| Method                                                | How                                                                                  |
|-------------------------------------------------------|--------------------------------------------------------------------------------------|
| --server option                                       | --server https://user:accesskey@example.com/teamscale/ (can repeat per server)       |
| TEAMSACLE_DEV_SERVERS env var                         | Same credentials-URL format, space-separated for multiple servers                    |
| --user / --accesskey options                          | --user NAME --accesskey KEY (applies to all servers not matched by --server)         |
| TEAMSACLE_DEV_USER / TEAMSACLE_DEV_ACCESSKEY env vars | Env equivalent of the above                                                          |
| Args file                                             | Put --server ... in ~/.teamscale-dev.args, then pass @"\${HOME}/.teamscale-dev.args" |

Recommended for terminal use – set once in your ~/.zshrc (you're on zsh/macOS):

```
export TEAMSACLE_DEV_SERVERS="https://username:accesskey@example.com/teamscale/"
```

Then you never need credential flags. ⚠️ Avoid passing --accesskey/--server directly on the command line in shared environments – other users can read process args (ps). Prefer env vars or the args file. Note @ in a username must be percent-escaped as %40 (though teamscale-dev auto-corrects @).

Verify it works

```
teamscale-dev verify-configuration src/ # validates .teamscale.toml + server/project access
```

Common commands once configured

- teamscale-dev pre-commit src/ – upload local changes for pre-commit analysis and show findings (add --upload-uncommitted-files when paths are under Git/SVN – recommended)
- teamscale-dev fetch-findings src/ – show findings already on the server (no upload)
- teamscale-dev open-in-browser src/ – open the file/dir in the Teamscale web UI
- teamscale-dev mcp – run as an MCP server for AI coding assistants
- teamscale-dev generate-completion – shell tab-completion (bash/zsh)

Other useful flags: --branch (override branch), --insecure or a #trust-all-certificates URL suffix (disable TLS validation – testing only), --debug/--trace (logging).

Want me to help you write a .teamscale.toml for a specific project or set up the zsh env vars?

The same pairing, on a tool outside either server

and why do i need user and accesskey then

- Good question – you don't need both. They're alternative ways to supply the same credentials, useful in different situations.

The distinction is how credentials get matched to a server URL:

- TEAMSCALE\_DEV\_SERVERS (and the --server flag) – credentials are bound to a specific server URL, because the username and access key are baked into the URL itself (https://user:key@example.com/...). This is per-server.
- TEAMSCALE\_DEV\_USER / TEAMSCALE\_DEV\_ACCESSKEY (and --user / --accesskey) – credentials that are not tied to any particular URL. They apply as a fallback to every server referenced in your .teamscale.toml files that doesn't already have credentials from a --server/TEAMSCALE\_DEV\_SERVERS entry.

So:

| Situation                                                  | What to use                                                  |
|------------------------------------------------------------|--------------------------------------------------------------|
| One server, or same login everywhere                       | Either one alone is enough – TEAMSCALE_DEV_SERVERS is fine   |
| Different logins for different servers                     | TEAMSCALE_DEV_SERVERS (each URL carries its own credentials) |
| One default login for whatever server the config points at | TEAMSCALE_DEV_USER + TEAMSCALE_DEV_ACCESSKEY                 |

If you've set TEAMSCALE\_DEV\_SERVERS for your server, you do not also need TEAMSCALE\_DEV\_USER/TEAMSCALE\_DEV\_ACCESSKEY. The --server form takes precedence anyway.

Why offer the --user/--accesskey form at all? Because it separates the URL (which lives in the shared, committed .teamscale.toml) from the credentials (which are personal). With it you can keep just server.url = "https://example.com/teamscale/" in the project config and let each developer supply their own user/key via env vars – no secrets in the URL, no secrets in version control.

So for your single-server case: pick one approach. TEAMSCALE\_DEV\_SERVERS alone is all you need.

## The honest ledger: **capability, not opinion**

### ✓ **Can**

- Expose the **entire REST API** as tools
- Serve **any MCP client** (Claude Code, CI, web, cron)
- Authenticate as **any identity** (personal or service)
- **Explain and act** together (with docs-mcp)
- **Observability**: every call through one vantage point
- **Retrofit** old instances

### ✗ **Can't**

- **No local working tree** → `local-*` stay on `teamscale-dev`
- **Raw tools, not workflows** → orchestration on the agent, so not the smoothest fix/PR loop

It's an addition, not a replacement for `ts_agent_helper`

## How they fit together

Sidecars next to Teamscale. Pick the pieces per audience:

- 📖 **teamscale-docs-mcp** on `:8082` (no auth, the manual)
- 🔌 **teamscale-mcp** on `:8081` (per-request identity, the data)
- 🧩 **The plugin** on top, optional (code workflows for developers)

Point any MCP client at both URLs. You get explain-and-act for everyone, and code editing for the devs who also want the plugin.

# Thank you

**teamscale-mcp** gives Teamscale to any agent and any identity.

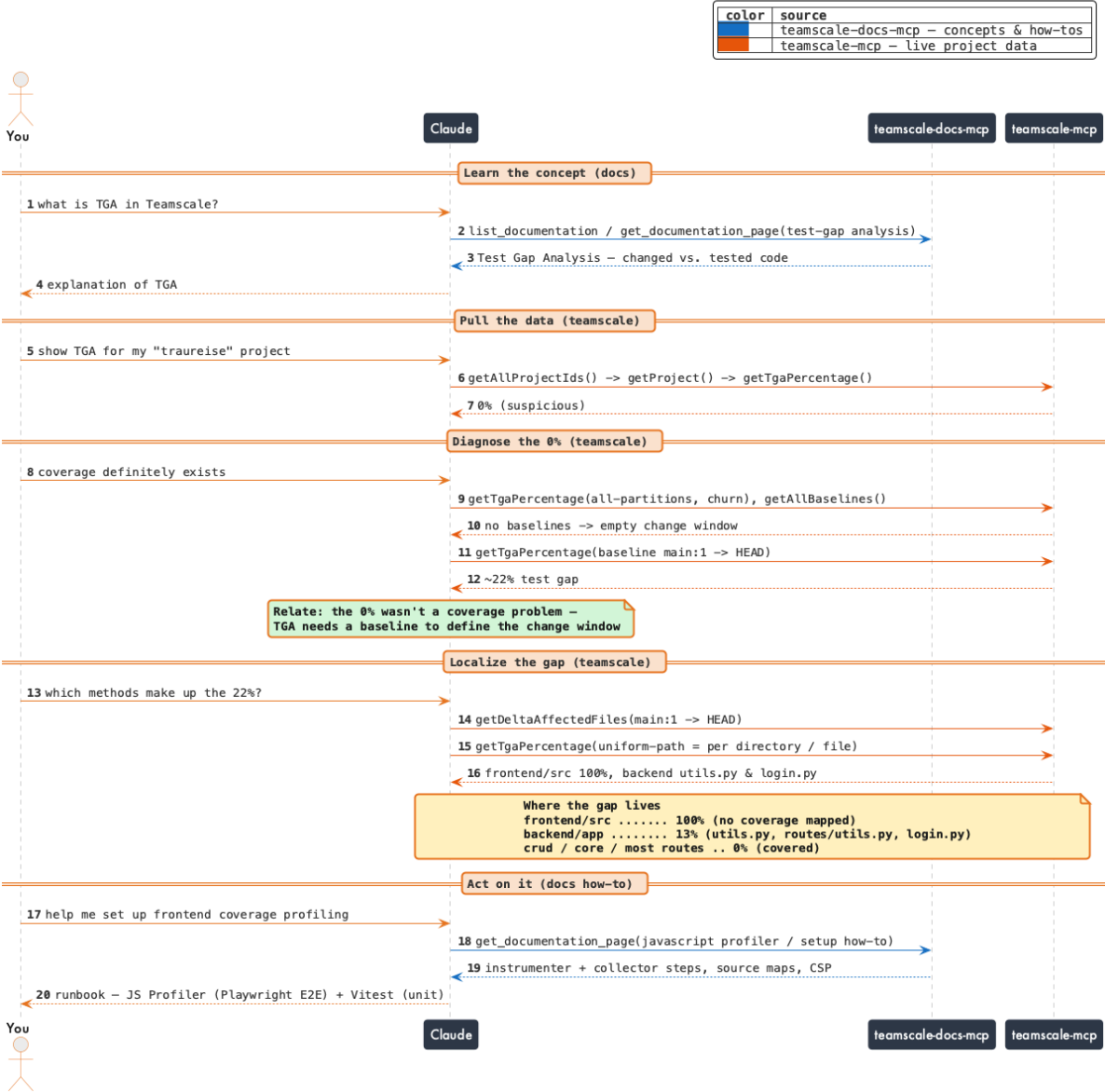
**teamscale-docs-mcp** gives it the manual.

*Questions?*



# Backup

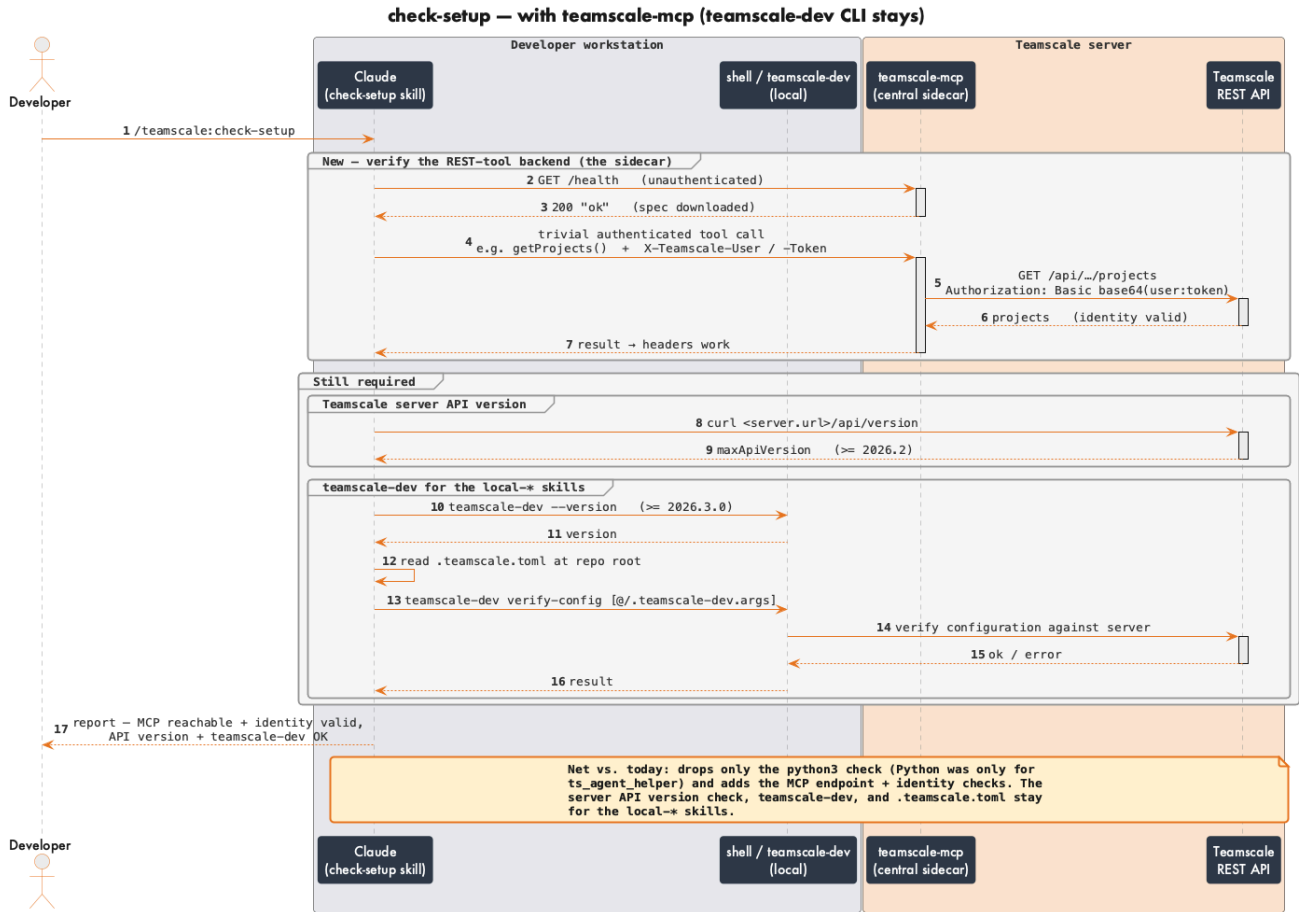
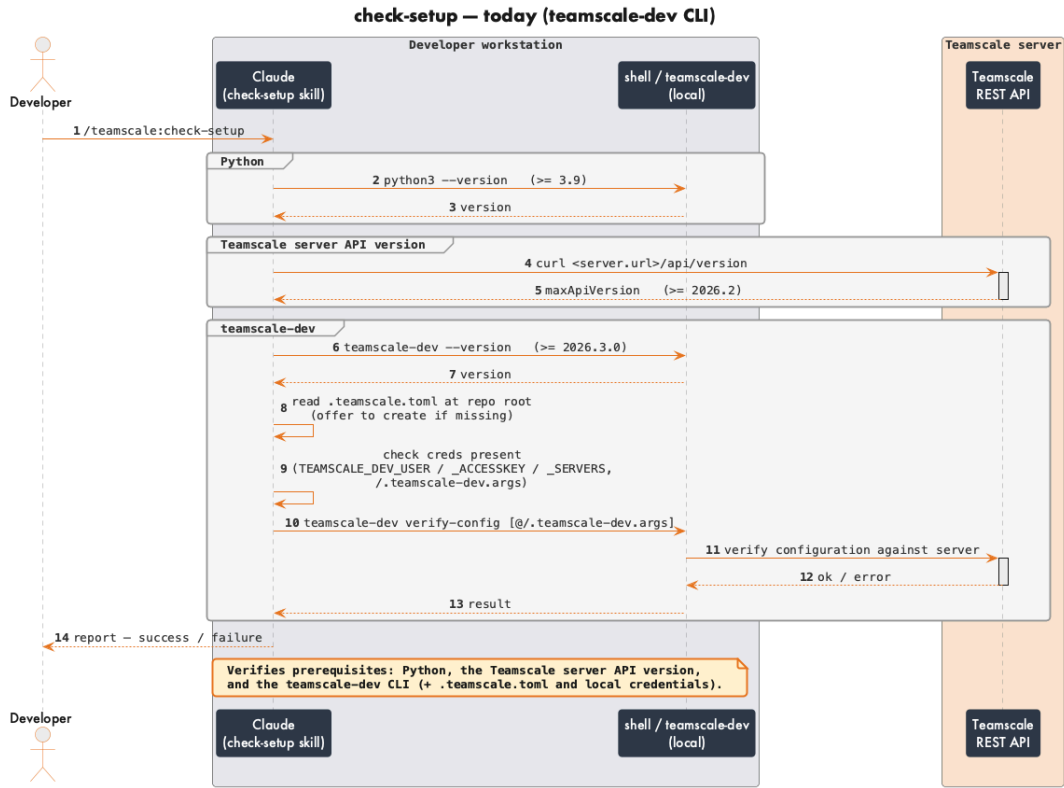
Check TGA & set up frontend profiling — you, teamscale-mcp & teamscale-docs-mcp

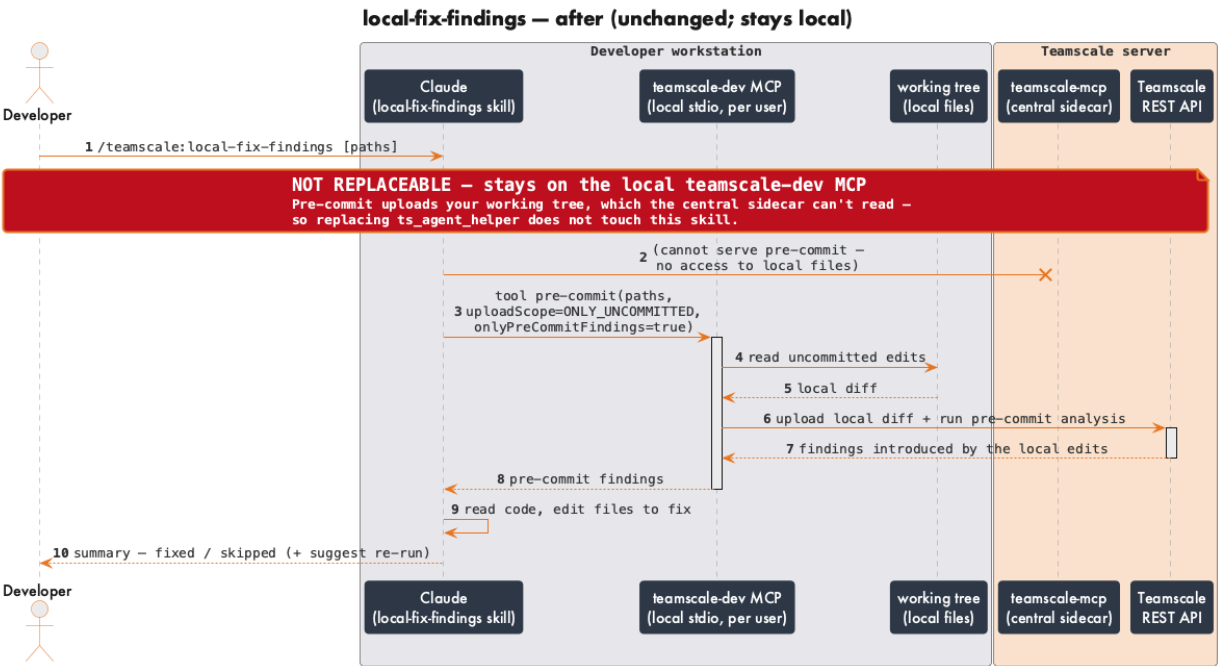
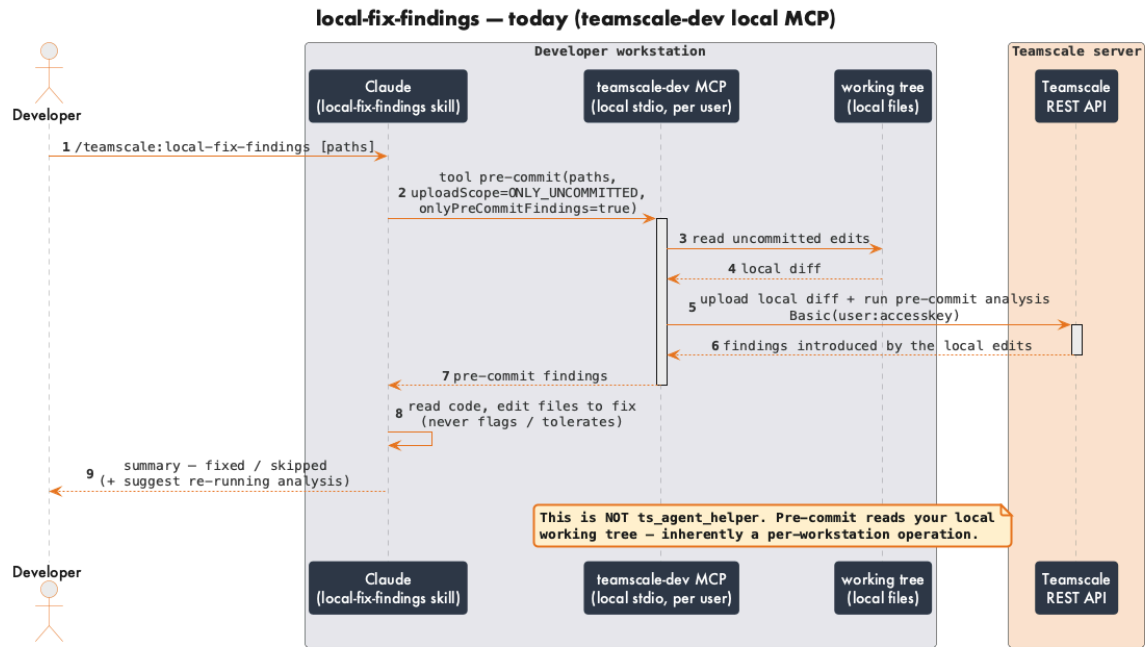


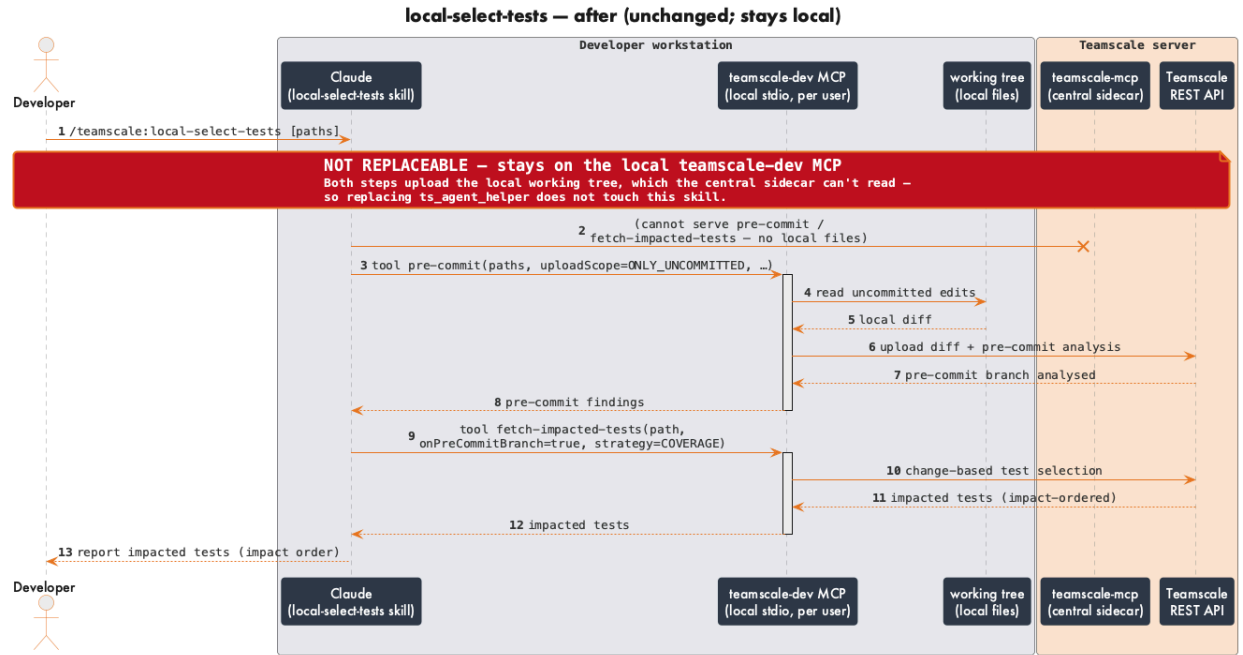
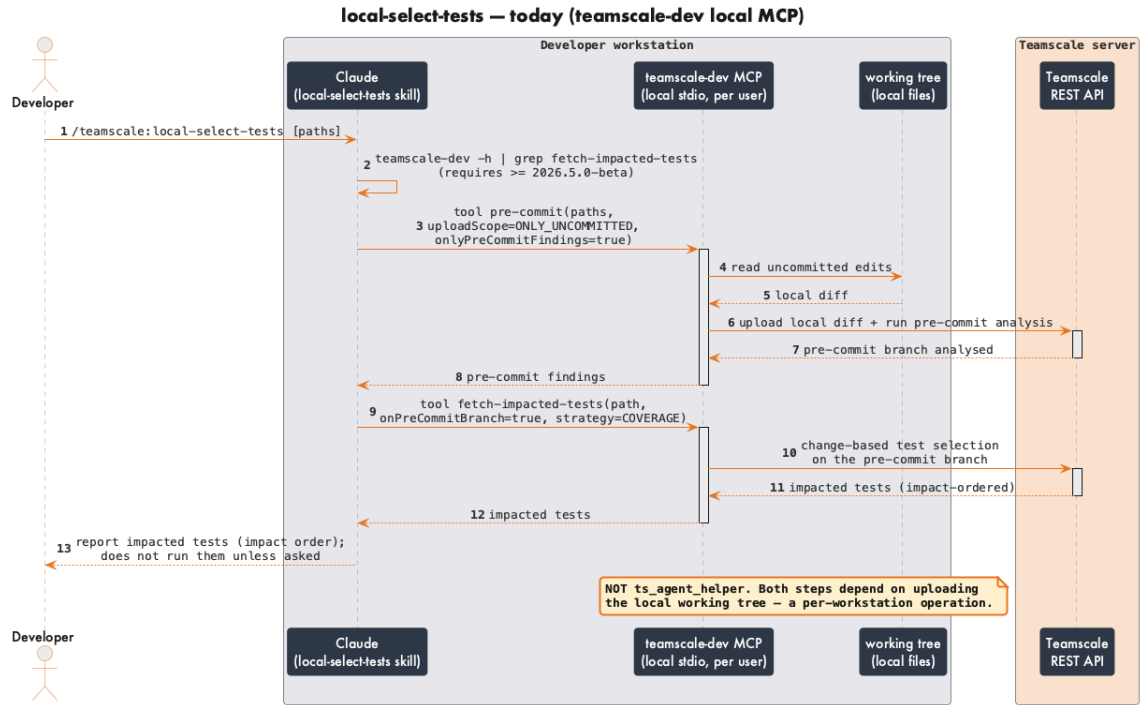
# Plugin comparison

Each diagram: **today** ( `ts_agent_helper` ) on the left, **with** `teamscale-mcp` on the right.

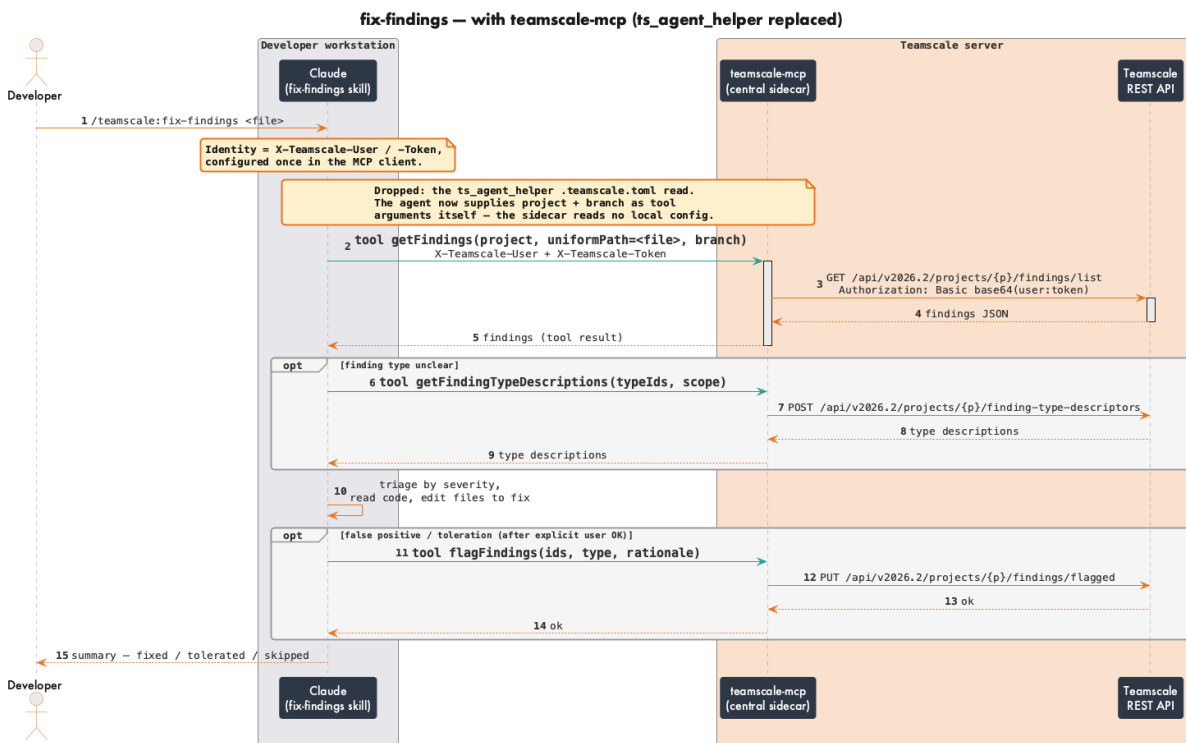
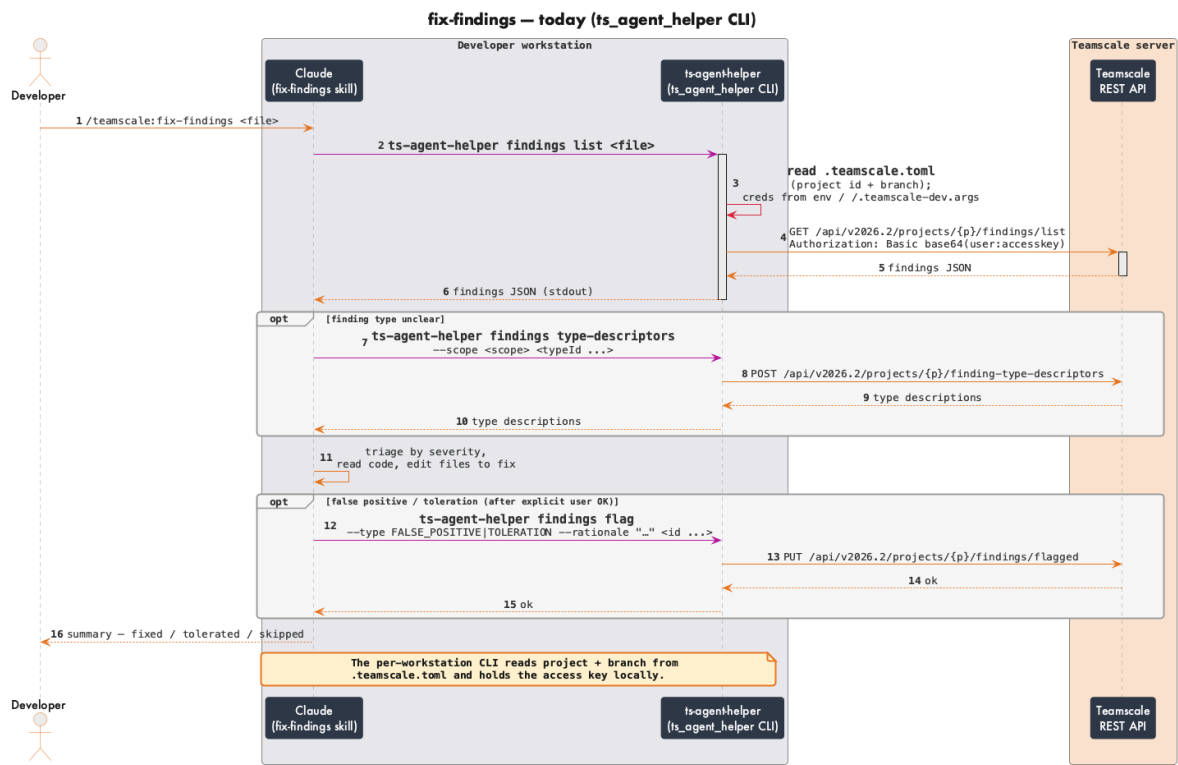
check-setup — shifts to verifying the endpoint



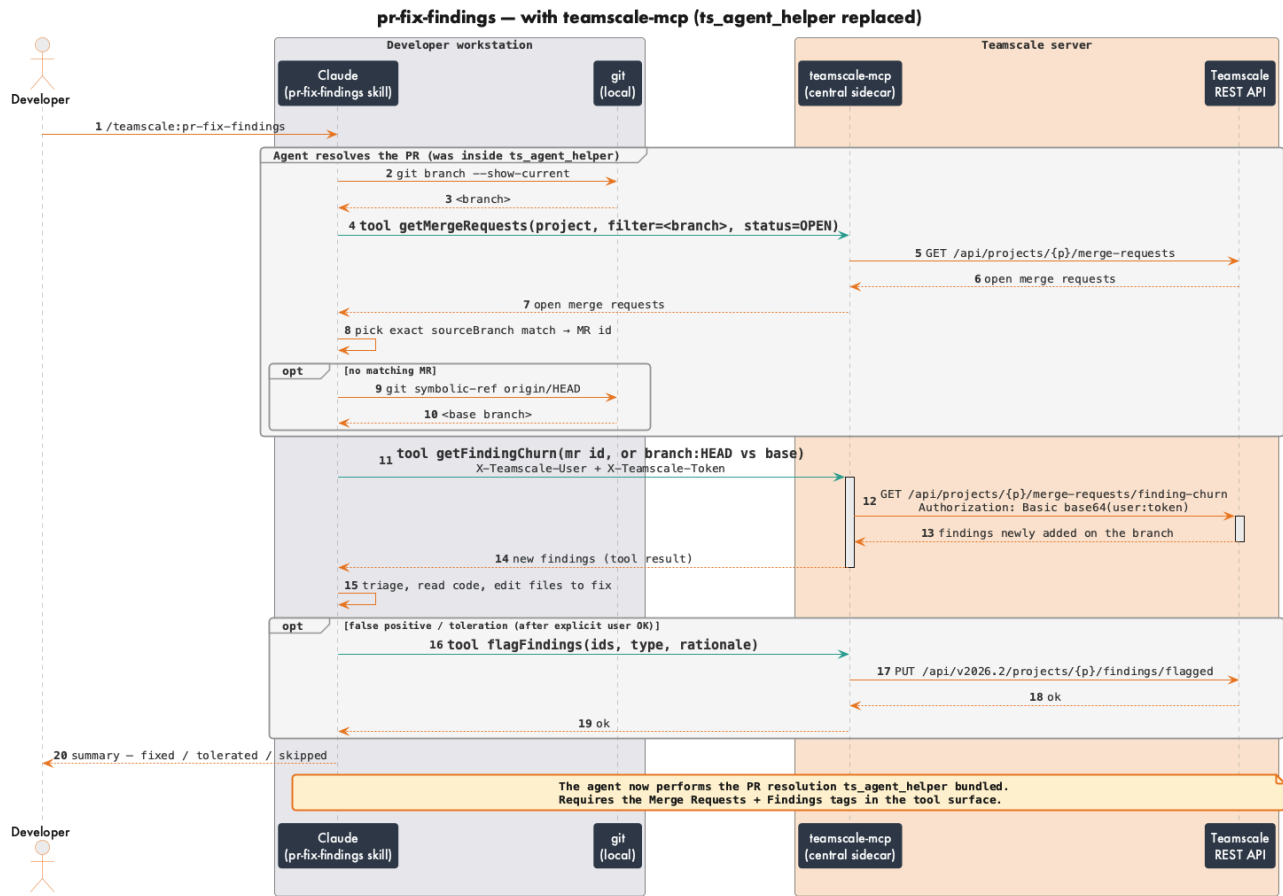
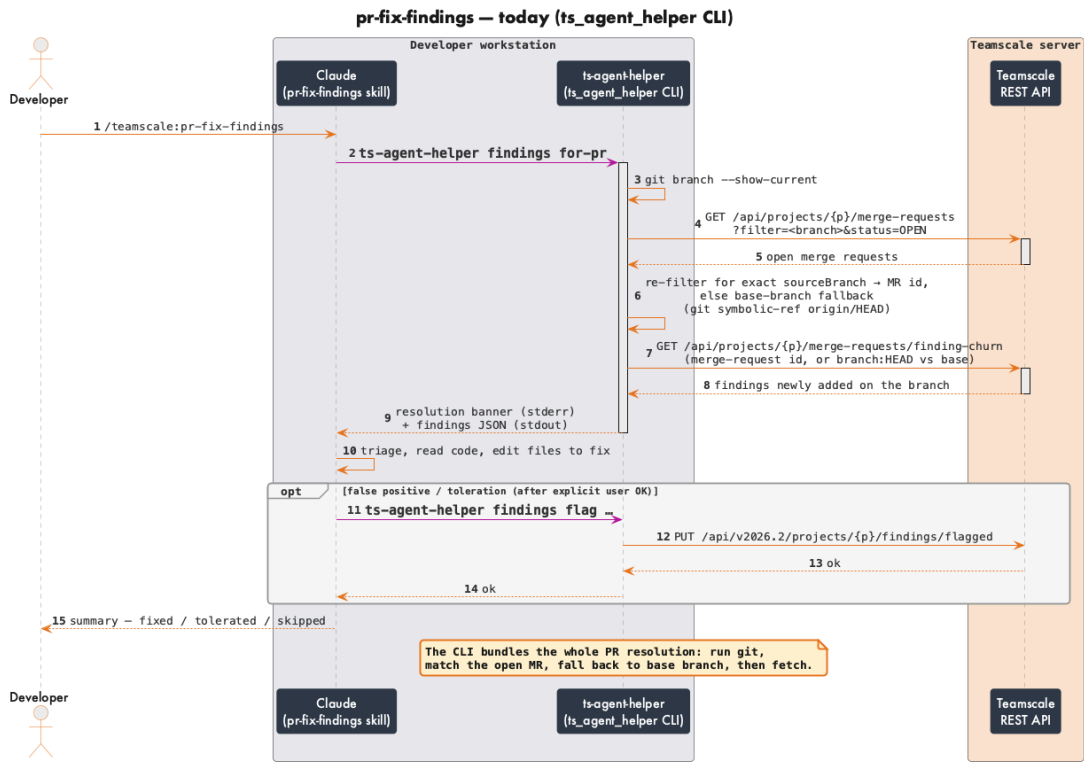




fix-findings — replaceable (clean win)



pr-fix-findings — replaceable (agent takes over PR resolution)



pr-close-test-gaps — replaceable (agent takes over PR resolution)

